Koneru Lakshmaiah Education Foundation
(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)
Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.

Academic Year 2026–2027

PROJECT REPORT

ON

"ARGUMENT PASSING"

A Project Report Submitted in partial fulfilment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

in

COMPUTER SCIENCE AND ENGINEERING

Course: Operating Systems and Systems Programming (25CS2104E)

Submitted by

Roll Number	Student Name
2520030156	Y.Ratna priya
2520030086	J.Hasini
2520030132	N.Teja Sri

Team No. 2 | Section 6

Under the Esteemed Guidance of

_____

Department of Computer Science and Engineering

Academic Year 2026–2027 | Term I

DECLARATION

We hereby declare that the project report entitled “Argument Passing” is a record of bona fide work carried out by us in partial fulfilment of the requirements for the award of the degree of Bachelor of Technology in Computer Science and Engineering at K L (Deemed to be) University, during the academic year 2026–2027.

The work presented in this report is based on the project problem statement prepared for the course Operating Systems and Systems Programming (25CS2104E). The report describes the proposed Linux-based argument-passing project. Implementation and execution evidence have not been represented as completed.

Roll Number	Student Name	Signature
2520030156	Y.Ratna priya	
2520030086	J.Hasini	
2520030132	N.Teja Sri	

CERTIFICATE

This is to certify that the project report entitled “Argument Passing” is a bona fide project report prepared by Y.Ratna priya (2520030156), J.Hasini (2520030086), and N.Teja Sri (2520030132), students of Section 6, Team No. 2, in partial fulfilment of the requirements for the award of the degree of Bachelor of Technology in Computer Science and Engineering at K L (Deemed to be) University, during the academic year 2026–2027.

The project report is based on the submitted project problem statement for Operating Systems and Systems Programming (25CS2104E).

Signature of the Guide	Signature of the Course Coordinator	Signature of the HOD
Name: _____	Name: _____	Name: _____

ACKNOWLEDGEMENT

We express our sincere gratitude to all those who supported and encouraged us during the preparation of the project report “Argument Passing”. The project problem statement helped us understand how command-line arguments can be handled in a Linux environment and how arguments may be passed from a parent process to a child process.

We extend our appreciation to our faculty guide, Course Coordinator, and the Head of the Department of Computer Science and Engineering for their academic guidance and support for the Operating Systems and Systems Programming course.

We are also thankful to our teaching and non-teaching staff members, parents, friends, and classmates for their encouragement and support.

2520030156 – Y.Ratna priya
2520030086 – J.Hasini
2520030132 – N.Teja Sri

ABSTRACT

Argument Passing is a system programming project that demonstrates how arguments can be passed from a parent process to a child process in a Linux environment. The project focuses on understanding how processes communicate and how command-line arguments are handled during program execution. It uses C programming and Linux process management mechanisms to create a simple and practical demonstration of argument passing.

The system accepts arguments from the command line and passes or makes them available to the required process for execution. It demonstrates the creation of processes, handling of process arguments, and execution of programs using Linux-based system programming concepts. The project helps in understanding process creation, command-line argument handling, and basic inter-process communication concepts.

The proposed solution will be implemented on Linux/Ubuntu using C programming and relevant POSIX/Linux APIs. The expected outcome is a working application that clearly demonstrates how arguments are received, processed, and passed between processes, providing a practical understanding of operating system and system programming concepts.

Keywords: Argument Passing, Linux, Ubuntu, C programming, command-line arguments, parent process, child process, fork(), exec(), POSIX APIs.

INDEX

S. No.	Chapter	Topics / Subtopics	Page
	Acknowledgement		i
	Abstract		ii
1	Introduction	1.1 Background; 1.2 Problem Statement; 1.3 Scope; 1.4 Objectives; 1.5 Importance; 1.6 Target Users	1
2	OS Concepts and Background Study	2.1 Concepts; 2.2 Existing Methods; 2.3 Proposed Approach; 2.4 Advantages	3
3	System Requirements	3.1 Hardware; 3.2 Software; 3.3 Ubuntu/Linux Environment; 3.4 Development Tools	5
4	Technology and Tools Used	4.1 C; 4.2 GCC; 4.3 Linux System Calls; 4.4 POSIX APIs; 4.5 GitHub	6
5	System Architecture and Design	5.1 Architecture; 5.2 Workflow; 5.3 Flowchart; 5.4 Module Description; 5.5 Input/Output Design	8
6	Implementation	6.1 Module-wise Plan; 6.2 Algorithm; 6.3 Argument Handling; 6.4 Process Creation; 6.5 APIs	11
7	Execution and Demonstration	7.1 Compilation; 7.2 Execution; 7.3 Expected Output; 7.4 Demonstration	14
8	Conclusion	8.1 Summary; 8.2 Objectives; 8.3 Expected Results; 8.4 Learning Outcomes	16
9	References	9.1 Books; 9.2 Linux Manuals; 9.3 GNU Documentation; 9.4 Online Resources	17

CHAPTER – 1 INTRODUCTION

1.1 Background of the Project

Linux is an operating system that supports multitasking and process management. A process is a program in execution. During program execution, a process may receive input through command-line arguments. These arguments allow users to provide information to a program when it is started, without requiring interactive input after launch.

In Linux system programming, a parent process can create a child process using fork(). A program can then be executed with arguments using functions from the exec() family. Understanding how arguments are received and passed helps students learn process creation, program execution, and basic Linux system programming.

1.2 Problem Statement

Argument Passing project aims to demonstrate how arguments are passed between processes in a Linux environment. It focuses on process creation, argument handling, and execution using C programming and Linux system calls.

1.3 Scope of the Project

The project scope is limited to a Linux/Ubuntu-based C program that accepts command-line arguments, processes the received arguments, and demonstrates how required arguments can be provided to a child process during execution. The project uses Linux/POSIX process-management mechanisms where required.

1.4 Objectives of the Project

1. To understand the concept of command-line argument passing in Linux.
2. To demonstrate how arguments are received and processed by a C program.
3. To implement argument passing between processes using Linux system programming mechanisms.
4. To provide a simple practical demonstration of process creation and argument handling.

1.5 Importance of the Application

The project provides a practical way to understand command-line arguments and the relationship between parent and child processes. It connects C programming with Linux process-management concepts and helps learners observe how arguments are supplied during program execution.

1.6 Target Users

The proposed application is intended for students learning Operating Systems, Linux system programming, C programming, process creation, and command-line argument handling.

CHAPTER – 2 OS CONCEPTS AND BACKGROUND STUDY

2.1 Operating System Concepts

Process Creation: Linux creates a new process using process-management mechanisms. The fork() system call creates a child process from a parent process.

Command-Line Arguments: Command-line arguments are values supplied when a program is started. In C, they are commonly received through argc and argv in the main() function.

Program Execution: The exec() family of functions replaces the current process image with a new program. Arguments can be specified as part of the execution call.

Process Management: Parent and child processes have separate process identities and can execute program instructions independently.

2.2 Existing Methods

A C program can receive command-line arguments directly when it is launched from a Linux terminal. This demonstrates argument reception by a single process. However, the project intends to extend the demonstration to include process creation and the provision of arguments to a child process.

2.3 Proposed Approach

The proposed approach uses a Linux/Ubuntu environment and C programming. The application accepts arguments through the command line. A parent process can create a child process using appropriate Linux process-management functions, and the required arguments can be provided to the child process during program execution.

2.4 Advantages

The approach provides a simple practical demonstration of command-line argument handling, process creation, parent-child process execution, and Linux/POSIX system programming functionality.

CHAPTER – 3 SYSTEM REQUIREMENTS

3.1 Hardware Requirements

A computer or laptop capable of running a Linux/Ubuntu environment is required. No special hardware device is specified in the submitted project problem statement.

3.2 Software Requirements

The planned software environment consists of Linux/Ubuntu, C/C++ programming tools, GCC compiler, POSIX APIs, and GitHub for source-code management and project repository hosting.

3.3 Ubuntu/Linux Environment

Linux/Ubuntu provides the execution environment for compiling and running the C program. The environment supports process creation and program execution through Linux system calls and POSIX interfaces.

3.4 Development Tools

Tool / Platform / Software	Purpose
Linux / Ubuntu	Development and execution environment
C / C++	Project implementation
GCC Compiler	Compilation of C programs
POSIX APIs	Process and system programming operations
GitHub	Source code management and project repository

CHAPTER – 4 TECHNOLOGY AND TOOLS USED

4.1 C Programming Language

C is the planned implementation language for the project. It provides access to command-line arguments through argc and argv and supports system-level programming through standard and POSIX interfaces.

4.2 GCC Compiler

GCC is used to compile C source programs into executable files that can be run in the Linux environment.

4.3 Linux System Calls

Linux system calls provide an interface between user programs and the operating-system kernel. The project uses process-related mechanisms to demonstrate process creation and execution.

4.4 POSIX APIs

POSIX APIs provide standard interfaces for process and system programming operations in a Linux-based environment.

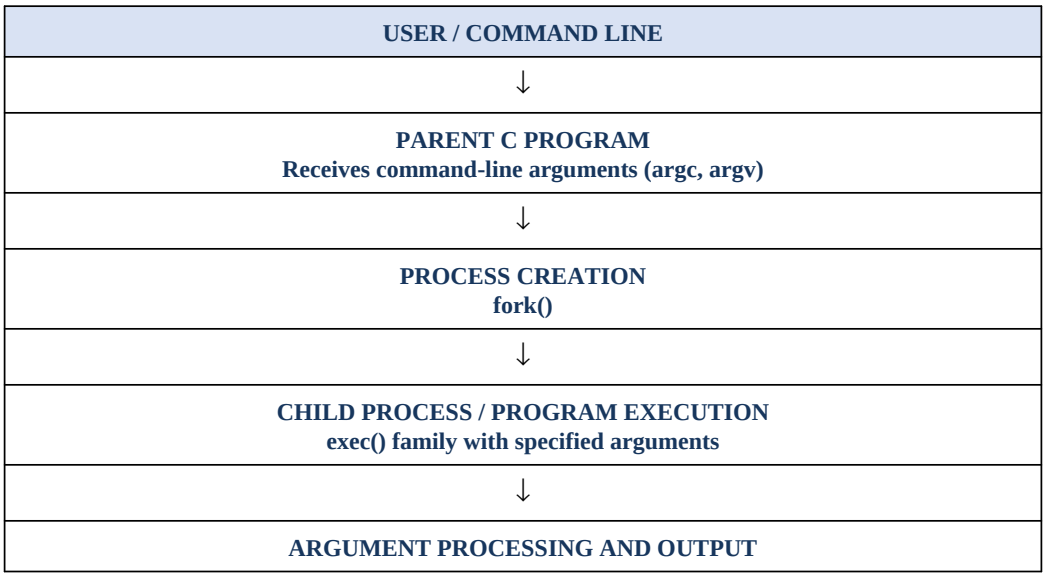
4.5 GitHub

GitHub is used for source-code management and project repository hosting. The repository link supplied in the project form is included in the References section.

CHAPTER – 5 SYSTEM ARCHITECTURE AND DESIGN

5.1 System Architecture

The proposed system follows a simple command-line architecture. User-supplied arguments are received by the parent program. The parent process handles the input and creates a child process when required. The required arguments are then made available to the child process for execution and processing.



5.2 Workflow

1. The user supplies arguments through the Linux command line.
2. The C program receives the arguments through argc and argv.
3. The parent process processes the received input.
4. The parent process creates a child process using fork() when required.
5. The required arguments are provided to the child process during execution.
6. The program displays the received and processed arguments.

5.3 Flow of Data

The data flow begins with command-line input, continues through argument reception and process creation, and ends with argument processing and output display.

5.4 Module Description

Argument Input Module: accepts values supplied through the command line.
Argument Processing Module: reads and processes the received arguments.
Process Creation Module: demonstrates parent and child process creation.
Execution Module: uses Linux program-execution mechanisms to run the required program with arguments.
Output Module: displays the arguments received and processed.

CHAPTER – 6 IMPLEMENTATION

6.1 Module-wise Implementation Plan

The project problem statement specifies a Linux/Ubuntu implementation using C programming. The planned modules are command-line argument reception, argument processing, parent-child process creation, execution with arguments, and output display.

6.2 Argument Reception

The C main() function can receive command-line arguments through argc and argv. argc represents the number of command-line arguments received, while argv stores the argument strings.

6.3 Process Creation

The fork() system call is used to create a child process. The parent and child processes can then follow separate execution paths. This mechanism is included to demonstrate parent-child process execution.

6.4 Program Execution with Arguments

The exec() family is used to execute a program with specified arguments. The proposed demonstration provides the required arguments to the program during execution.

6.5 Operating Systems Concepts / Linux APIs Used

OS Concept / Linux API / System Call	Purpose in the Project
Process Creation	To create and manage processes
Command-Line Arguments	To receive input arguments during program execution
fork()	To create a child process
exec() family	To execute a program with specified arguments
Process Management	To demonstrate parent and child process execution
POSIX APIs	To implement Linux-based system programming functionality

6.6 Implementation Status

The supplied project documentation describes the proposed methodology and expected outcome. It does not provide completed source code or verified execution screenshots. Therefore, this report does not claim that implementation or testing has already been completed.

CHAPTER – 7 EXECUTION AND DEMONSTRATION

7.1 Planned Compilation

The C source program is planned to be compiled in Linux/Ubuntu using the GCC compiler. The executable will then be run from the terminal with command-line arguments.

7.2 Planned Execution

During execution, the program will accept arguments from the command line. The parent process can create a child process using process-management functions, and the required arguments can be supplied to the child process during execution.

7.3 Expected Output

The expected output will display the arguments received and processed by the program. It is intended to make the argument-passing mechanism clear to the user.

7.4 Demonstration Status

No execution screenshots or verified output were included in the supplied project problem statement. Actual screenshots and output should be inserted after the program has been implemented and executed successfully.

CHAPTER – 8 CONCLUSION

8.1 Summary

Argument Passing is a Linux system programming project designed to demonstrate how command-line arguments are received, processed, and passed between processes. It focuses on C programming, process creation, parent-child process execution, and Linux/POSIX mechanisms.

8.2 Objectives

The project objectives are to understand command-line argument passing in Linux, demonstrate argument reception and processing in C, implement argument passing between processes using Linux system programming mechanisms, and provide a practical demonstration of process creation and argument handling.

8.3 Expected Results

The expected outcome is a working Linux-based application that accepts command-line arguments, processes them, and passes the required arguments to the appropriate process or program.

8.4 Learning Outcomes

The project is expected to provide practical understanding of process creation, command-line arguments, process execution, and Linux system programming mechanisms.

CHAPTER – 9 REFERENCES

9.1 Books

1. Operating Systems and Systems Programming course material (25CS2104E).

9.2 Linux Manuals

1. Linux manual pages for fork() and exec() family functions.

9.3 Programming and System Documentation

1. GNU Compiler Collection (GCC) documentation.
2. POSIX/Linux system programming API documentation.

9.4 Project Repository

GitHub Repository Link: https://github.com/yratnapriya05/2520030156_86_132_OS_PROJECT
Repository Created: YES

PROJECT DETAILS AND INDIVIDUAL CONTRIBUTION

Individual Contribution

Roll Number	Student Name	Individual Responsibility
2520030156	Y.Ratna priya	Project design and argument-passing implementation and Documentation and GitHub management
2520030086	J.Hasini	Process creation and Linux system calls
2520030132	N.Teja Sri	Testing and output verification

Tools / Platforms / Software Used

Tool / Platform / Software	Purpose
Linux / Ubuntu	Development and execution environment
C / C++	Project implementation
GCC Compiler	Compilation of C programs
POSIX APIs	Process and system programming operations
GitHub	Source code management and project repository

GitHub Repository

https://github.com/yratnapriya05/2520030156_86_132_OS_PROJECT

Faculty Name and Signature

Faculty Name: _____
Signature: _____
Remarks: _____